OSAIMA

AI-Driven Context-Aware Operating System Interface

PRE-DESIGN REVIEW (PDR) — DETAILED TECHNICAL REPORT

Jyothy Institute of Technology
Department of Artificial Intelligence and Machine Learning

Team Member	Role	Domain
Akash Manjunath	Lead Architect	System Integration & LLM Pipeline
Sunil J Rathore	Backend Developer	Linux Kernel & D-Bus Integration
Pragna S Bharadwaj	AI/ML Engineer	RAG Engine & Behavior Learning
Banu B B	Systems & Security	AppArmor, Sandboxing, Audit

Final Year Project | Academic Year 2024–2025
Document Version 1.0 | March 2025

1. Executive Summary

OSAIMA (OS AI Management & Automation) is a voice-first, AI-driven operating system interface built natively on Linux. It replaces the traditional GUI-and-CLI paradigm with a generative-AI agent that listens, reasons, and acts — all on-device. The system combines five critical technologies that no existing Linux voice assistant has unified: on-device automatic speech recognition (ASR) via Whisper/Vosk, local large language model (LLM) inference via Ollama/llama.cpp, OS-level desktop control via D-Bus and AT-SPI, retrieval-augmented generation (RAG) for contextual knowledge, and behavior-learning to adapt to individual user workflows.

The research paper accompanying this project (surveying 21 systems across 7 capability dimensions) confirms that OSAIMA is the only proposed system combining all five properties. The global voice-assistant market is projected at USD 33.74 billion by 2030 (CAGR 26.5%), yet Linux desktop users — who constitute 40–55% of professional developers — remain completely underserved. OSAIMA targets precisely this gap.

> *Mission: Deliver a privacy-preserving, fully local, voice-first Linux OS interface where an on-device LLM agent is the primary interaction primitive — not a widget, not a shortcut — but the OS shell itself.*

2. Problem Analysis & Motivation

2.1 The Six Structural Gaps in Existing Systems

Based on the literature survey of 21 systems, six critical gaps exist that OSAIMA directly addresses:

- **Gap 1 — No Voice-First Linux Distribution: Every existing system treats voice as an add-on layer atop a GUI-primary OS. No Linux distro boots voice-first by design.**
- **Gap 2 — No LLM-Driven Workflow Generation: Production platforms (Mycroft, Rhasspy) use hand-authored skill functions. They cannot synthesize novel multi-step workflows like 'set up a Python project with my usual config and push to GitHub.'**
- **Gap 3 — No Behavior-Learning Layer: SEPIA's simple key-value preferences aside, zero surveyed systems learn from user behavior — email style, commit conventions, code scaffold preferences — and inject them into LLM prompts.**
- **Gap 4 — Fragmented Desktop-Environment Integration: pocketVox targets only GNOME; Rhasspy targets IoT/MQTT. No unified voice control layer spans GNOME, KDE Plasma, Sway, and Wayland.**
- **Gap 5 — Weak Security for LLM-Generated OS Actions: No surveyed system implements risk-tiered confirmation, systemd sandboxing, or AppArmor confinement for its action-execution layer. Prompt injection has zero defences.**
- **Gap 6 — No Style-Template Storage: No system captures user writing style, commit message format, or code scaffold conventions and applies them to LLM-generated outputs.**

2.2 Why Mycroft Failed — And What OSAIMA Learns From It

Mycroft AI, the most complete prior attempt, was archived in 2023 due to commercial insolvency — but the root cause was architectural. Every new capability required a hand-authored Python skill. When users expected GPT-class open-ended reasoning, Mycroft had no path to deliver it. OSAIMA avoids this ceiling by making LLM inference the core reasoning engine rather than a plugin.

2.3 Market Opportunity

Metric	Value	Source
Global Voice Assistant Market (2024)	USD 7.35 billion	Market Research Future, 2024
Global Voice Assistant Market (2030)	USD 33.74 billion	Market Research Future, 2024
CAGR (2024–2030)	26.5%	Market Research Future, 2024
Linux Desktop Share (Professional Devs)	40–55%	Stack Overflow Dev Survey 2023
AI-in-Voice Sub-segment Growth	Highest CAGR in segment	MarketsandMarkets, 2024
AI Coding Productivity Gain (GitHub Copilot)	55.8% faster task completion	GitHub Research, 2023

3. System Architecture — Complete Technical Design

OSAIMA's architecture consists of six tightly integrated subsystems, each a systemd user service, communicating over Unix domain sockets with a structured JSON message protocol.

3.1 Architecture Layer Diagram (Textual)

[USER VOICE / TEXT INPUT] → [ASR DAEMON (Vosk/Whisper)] → [INTENT ROUTER (BERT-tiny / keyword)] → [LLM ENGINE (Ollama / llama.cpp)] → [ACTION EXECUTOR (D-Bus / AT-SPI / PTY / LSP)] → [DESKTOP / OS ACTIONS]

↑ RAG retrieval ↑ ↑ Behavior Store ↑

3.2 Subsystem 1 — ASR Daemon (Speech-to-Text)

Technology Stack

- Primary (CPU-only): Vosk large model (1.8 GB) — 95% WER on developer vocabulary, ~500ms latency on modern CPU
- Primary (GPU): OpenAI Whisper large-v3 — state-of-the-art WER, 0.8–2.1s latency on RTX 3060
- Wake-word: openWakeWord (Hey OSAIMA) — ~5–15% CPU overhead, runs continuously as ring-0 user daemon
- Audio subsystem: PipeWire pw-record API — replaces PulseAudio, lower latency, better Wayland compatibility
- VAD (Voice Activity Detection): WebRTC-based VAD to segment utterances before STT — reduces hallucinations

Implementation

The ASR daemon runs as a systemd user service (osaima-asr.service). It opens a continuous stream on the default PipeWire source, applies WebRTC VAD, passes confirmed utterance chunks to Vosk's streaming RecognizerStream, and emits finalized transcripts to a Unix domain socket at /run/user/$UID/osaima/asr.sock. Whisper is invoked as a fallback when Vosk confidence < 0.75.

How to Build — ASR Daemon

- pip install vosk pyaudio webrtcvad sounddevice --break-system-packages
- Download Vosk model: wget https://alphacephei.com/vosk/models/vosk-model-en-us-0.22.zip
- For Whisper GPU: pip install openai-whisper torch torchaudio --break-system-packages
- For openWakeWord: pip install openwakeword --break-system-packages
- Wrap in asyncio event loop; emit JSON: {transcript, confidence, timestamp} over Unix socket

3.3 Subsystem 2 — Intent Router

Design Philosophy

The intent router acts as a traffic cop: it must decide in under 200ms whether an utterance needs the LLM (slow path) or can be handled by a direct OS call (fast path). This keeps simple commands (open Firefox, mute audio) snappy while routing complex requests (set up a Django project with my usual settings) to the LLM.

Three-Track Dispatch

Track	Latency Target	Handler	Example
System Command	< 200ms	Direct D-Bus / subprocess call	'Open Firefox', 'Mute audio', 'Lock screen'
Workflow Generation	< 8 sec	LLM inference engine	'Set up Python project with my config'
Content Generation	< 12 sec	LLM inference engine	'Draft an email to my team about the deadline'

Implementation

- Stage 1: Keyword matcher (regex + trie) handles ~200 common Linux commands sub-50ms
- Stage 2: Fine-tuned BERT-tiny (Hugging Face transformers) classifies utterance into 3 tracks + 40 system intents
- Training data: Synthesize 10,000 utterances using the LLM itself; label manually for 500 edge cases
- Model size: ~15MB; inference ~30ms on CPU — fits within latency budget
- pip install transformers torch sentence-transformers --break-system-packages

3.4 Subsystem 3 — Local LLM Inference Engine

Model Selection

Model	Size (Q4_K_M)	RAM	Speed (CPU)	Recommended For
Llama-3.1-8B-Instruct	4.9 GB	6 GB	15–25 tok/s	Primary — best instruction following
Mistral-7B-Instruct-v0.3	4.1 GB	5.5 GB	18–28 tok/s	Fallback — faster on weak CPUs
Phi-3-mini-4k-Instruct	2.2 GB	3.5 GB	30–45 tok/s	Low-resource devices (4 GB RAM)
Llama-3.1-70B-Instruct	40 GB	48 GB	2–5 tok/s	High-end workstations, max quality

Inference Backend

- Primary: Ollama — REST API on localhost:11434, automatic GPU/CPU routing, streaming responses
- Alternative: llama-cpp-python — direct Python bindings, zero extra process, lower overhead for embedded targets
- Install Ollama: curl -fsSL https://ollama.ai/install.sh | sh
- Pull model: ollama pull llama3.1:8b-instruct-q4_K_M
- Python: from ollama import Client; client = Client(); stream = client.chat(model=..., messages=..., stream=True)

System Prompt Architecture

Every LLM call receives a structured system prompt containing: (1) Current working directory and active application context, (2) User behavior profile from the behavior store (writing style, scaffold

preferences, commit conventions), (3) Tool-call schema describing available OS actions as JSON-schema function definitions, (4) RAG-retrieved context from the knowledge base relevant to the utterance, (5) Safety rules: 'Never issue destructive commands without user confirmation; classify every action as read-only / reversible / irreversible.'

3.5 Subsystem 4 — RAG Engine (Retrieval-Augmented Generation)

The RAG engine provides the LLM with relevant contextual knowledge without retraining. It indexes three knowledge sources: (1) the user's personal documentation and notes, (2) system manuals and man pages, and (3) project-specific READMEs and codebases.

Technical Stack

- Embedding model: sentence-transformers/all-MiniLM-L6-v2 (80 MB, CPU-viable, 384-dim embeddings)
- Vector store: ChromaDB (local SQLite backend) — pip install chromadb --break-system-packages
- Document ingestion: LangChain loaders for PDF, Markdown, text, and source code — pip install langchain langchain-community --break-system-packages
- Retrieval: Top-5 cosine similarity chunks, injected into LLM context at inference time
- Re-ranking: Cross-encoder (ms-marco-MiniLM-L-6-v2) scores retrieved chunks against query for precision

How to Build RAG

- from chromadb import Client; from langchain.text_splitter import RecursiveCharacterTextSplitter
- Split documents into 512-token chunks with 50-token overlap
- Embed with SentenceTransformer; store embedding + metadata in ChromaDB collection
- At query time: embed utterance → retrieve top-5 → re-rank → inject into prompt

3.6 Subsystem 5 — OS-Level Action Execution

This is the most technically novel subsystem — the bridge between LLM output and actual OS actions. It operates through four integration interfaces:

Interface A — D-Bus (Primary Desktop Control)

- GNOME Shell: org.gnome.Shell D-Bus interface — application launching, extension control, notification management
- Media control: MPRIS2 (org.mpris.MediaPlayer2) — play/pause/next/volume for any media player
- Network: org.freedesktop.NetworkManager — connect/disconnect WiFi, VPN, Bluetooth
- Services: org.freedesktop.systemd1 — start/stop/status of any systemd service
- Python library: python-dbus or dbus-python; pip install dbus-python --break-system-packages
- Example: bus.get_object('org.gnome.Shell', '/org/gnome/Shell').Eval('Main.overview.show()', dbus_interface='org.gnome.Shell')

Interface B — AT-SPI (Application Widget Interaction)

- AT-SPI2 (Assistive Technology Service Provider Interface) provides structured access to the full widget tree of all running GTK4 and Qt6 apps
- Enables: filling text fields, clicking buttons, reading table contents, selecting menu items — all from voice
- Python: import pyatspi; desktop = pyatspi.Registry.getDesktop(0)
- pip install pyatspi --break-system-packages; enable AT-SPI: gsettings set org.gnome.desktop.interface toolkit-accessibility true
- This is how Orca (GNOME screen reader) drives all Linux applications — proven, battle-tested API

Interface C — PTY Injection (Terminal Automation)

- For terminal emulators (GNOME Terminal, Kitty, Alacritty): inject commands into the active PTY using the terminal's D-Bus interface or /proc/PID/fd manipulation
- Python library: pexpect — high-level PTY control; import pexpect; child = pexpect.spawn('/bin/bash')
- More powerful: use python-ptyprocess for raw PTY creation; mount the assistant's Python process as a co-process in the shell session
- Enables: 'run the tests', 'activate the venv', 'git commit with my usual message format' — executed directly in the open terminal

Interface D — LSP (IDE-Level Code Actions)

- Language Server Protocol clients allow the assistant to: refactor functions, insert snippets, trigger code completion, and navigate symbols inside active IDE sessions
- Libraries: pylsp-jsonrpc (Python LSP client), or drive VS Code via its JSON-RPC extension API over WebSocket
- Enables: 'refactor this function', 'add type hints to all parameters', 'generate docstring for this class'

3.7 Subsystem 6 — Behavior Store & Style-Template Engine

The behavior store is OSAIMA's memory — it learns who you are and how you work. It solves Gap 3 and Gap 6 from the literature survey.

Data Model

Store Component	Technology	What It Learns
Interaction Log	SQLite (append-only)	Every utterance, action taken, outcome, correction
Style Templates	SQLite + JSON blobs	Email tone, commit message format, code scaffold conventions
Workflow Sequences	SQLite graph table	Common multi-step patterns (e.g., git add → commit → push)
Project Profiles	ChromaDB collection	Per-project README, conventions, directory structure
Frequency Heuristics	SQLite counters	Most-used apps, commands, time-of-day patterns

Style Learning Pipeline

- After each LLM-generated output (email draft, commit message, code), user edits are diff'd against the original
- Diffs are summarized by the LLM into natural-language style rules: 'User prefers concise bullet points over prose in emails'
- Style rules are stored as embeddings in ChromaDB; retrieved at inference time via cosine similarity to current task
- Retrieved rules are injected into the system prompt: 'Apply these user style preferences to your output: [rules]'

4. Security Architecture — Gap 5 Resolution

Executing LLM-generated OS commands introduces a novel threat model that no prior Linux voice system has addressed. OSAIMA implements a four-layer security architecture:

4.1 Risk Classification

Risk Tier	Examples	Confirmation Required	Rollback?
Read-Only	cat file, ls, ps, df, check network status	None — auto-execute	N/A
Reversible	move file, install package, create directory	Single-click dialog	Yes — undo queue
Irreversible	rm -rf, format disk, send email, git push	Type-to-confirm + preview	Best-effort

4.2 systemd Execution Sandbox

Every LLM-generated shell script executes inside a systemd transient scope with capability restrictions:

```
systemd-run --scope --user --property=CapabilityBoundingSet='' --
property=PrivateTmp=yes --property=NoNewPrivileges=yes -- bash -c
"<GENERATED_SCRIPT>"
```

- CapabilityBoundingSet='' drops all Linux capabilities from the process
- PrivateTmp=yes creates an isolated /tmp namespace
- NoNewPrivileges=yes prevents privilege escalation via setuid binaries
- Further restrict with: ProtectHome=read-only, ProtectSystem=strict, RestrictAddressFamilies=AF_UNIX AF_INET

4.3 AppArmor Confinement

The OSAIMA assistant daemon itself runs under a custom AppArmor profile that limits what it can access, reducing the blast radius of any prompt injection attack:

```
/usr/bin/osaima-daemon {\n  network inet stream,\n  /home/@{user}/.config/osaima/
r,\n  /run/user/@{uid}/osaima/ rw,\n  deny /etc/passwd rw,\n  deny /etc/shadow rw,\n}
```

4.4 Prompt Injection Defence

- Context isolation: External document content (emails, web pages, clipboard) is tagged as UNTRUSTED_CONTEXT and stripped of instruction-like patterns before LLM injection
- Output validation: LLM-generated shell commands are parsed against an AST whitelist before execution; any syscall to /etc/, /boot/, or kernel interfaces requires user escalation
- Instruction boundary enforcement: System prompt uses a canary token sequence; if the canary appears in LLM output, the response is rejected as injection-compromised
- Tamper-evident audit log: SHA-256 chained append-only log records every utterance, LLM response, action taken, and user confirmation

5. Complete Build Guide — How to Construct OSAIMA

5.1 Base System Selection

Distribution	Recommended For	Audio	AppArmor/SELinux	Notes
Ubuntu 24.04 LTS	Primary target — developer users	PipeWire (default)	AppArmor (built-in)	5-yr LTS, best hardware support
Fedora 40	Enterprise & SELinux environments	PipeWire (default)	SELinux (default)	Cutting-edge packages
Arch Linux	Rolling-release development	PipeWire (manual)	AppArmor (opt-in)	Bleeding-edge llama.cpp/Whisper
Debian 12	Minimal/embedded targets	PipeWire or PulseAudio	AppArmor (opt-in)	Low-resource devices

5.2 Phase 1 — Environment Setup (Week 1–2)

1. Install Ubuntu 24.04 LTS minimal server; add GNOME Shell: sudo apt install gnome-shell gnome-terminal
2. Install PipeWire stack: sudo apt install pipewire pipewire-audio-client-libraries wireplumber
3. Install Python 3.11+: sudo apt install python3.11 python3-pip python3-venv
4. Install Ollama: curl -fsSL https://ollama.com/install.sh | sh && ollama pull llama3.1:8b-instruct-q4_K_M
5. Install Node.js 20 LTS for MCP servers: curl -fsSL https://deb.nodesource.com/setup_20.x | sudo bash -
6. Install system dependencies: sudo apt install libdbus-1-dev libgirepository1.0-dev python3-gi at-spi2-core

5.3 Phase 2 — ASR Daemon (Week 3–4)

7. pip install vosk sounddevice webrtcvad numpy --break-system-packages
8. Download Vosk model: vosk-model-en-us-0.22 (~1.8 GB) to ~/.local/share/osaima/models/
9. Build asr_daemon.py: asyncio loop reading PipeWire PCM stream → WebRTC VAD → Vosk KaldiRecognizer
10. Expose transcript stream on Unix socket: /run/user/$UID/osaima/asr.sock
11. Register systemd user service: ~/.config/systemd/user/osaima-asr.service

12. For wake-word: pip install openwakeword --break-system-packages; train custom model on 'Hey OSAIMA' (50 positive samples minimum)

5.4 Phase 3 — Intent Router (Week 5–6)

13. pip install transformers sentence-transformers scikit-learn --break-system-packages
14. Build keyword trie from 200 common Linux command patterns; test < 50ms on 8-core CPU
15. Fine-tune BERT-tiny on synthetic dataset: generate 10,000 utterances using Ollama, label with 3-track + 40 intent taxonomy
16. Export model to ONNX for faster CPU inference: model.save_pretrained(); optimum-cli export onnx
17. Build dispatcher: keyword_match() → if SYSTEM_CMD: d_bus_dispatch() elif WORKFLOW/CONTENT: llm_dispatch()

5.5 Phase 4 — RAG Engine (Week 7–8)

18. pip install chromadb langchain sentence-transformers --break-system-packages
19. Build indexer: watch ~/.local/share/osaima/knowledge/ for new documents; auto-ingest on file creation
20. Implement recursive splitter: 512-token chunks, 50-token overlap, preserve code blocks intact
21. Embed with all-MiniLM-L6-v2 (~80 MB); store in ChromaDB local collection
22. Build retriever: embed query → cosine search top-10 → re-rank with cross-encoder → return top-5
23. Index system: man pages (parse with pandoc), project README files, user notes in ~/Documents/

5.6 Phase 5 — Action Executor (Week 9–11)

D-Bus Integration

24. pip install dbus-python pydbus --break-system-packages
25. Build action registry: JSON file mapping intent names to D-Bus method calls with parameter schemas
26. Implement: AppLauncher (gio launch), WindowManager (GNOME Shell eval), MediaController (MPRIS2), NetworkManager, SystemdManager

AT-SPI Integration

27. pip install pyatspi --break-system-packages; gsettings set org.gnome.desktop.interface toolkit-accessibility true
28. Build UIAutomator class: find_window(title), find_element(role, name), click_element(), type_into_field()
29. Test with GTK4 apps: Nautilus, gedit, GNOME Settings — validate cross-app widget interaction

PTY & LSP

30. pip install pexpect ptyprocess --break-system-packages; build TerminalInjector class
31. For VS Code LSP: use python-language-server JSON-RPC; for Neovim: use pynvim remote API

5.7 Phase 6 — Behavior Store (Week 12–13)

32. Initialize SQLite DB at ~/.local/share/osaima/behavior.db with schema: interactions, style_rules, workflows, app_frequency
33. Build InteractionLogger: log every utterance + action + outcome + user_correction asynchronously
34. Build StyleLearner: after each content generation, diff user edits → summarize diff with LLM → embed rule → store in ChromaDB
35. Build WorkflowDetector: detect repeated action sequences (edit distance clustering); surface as automatable workflows

5.8 Phase 7 — Security Layer (Week 14)

36. Write AppArmor profile for osaima-daemon; test with aa-complain mode first, then enforce
37. Build RiskClassifier: parse LLM-generated commands through AST analyzer; assign Read/Reversible/Irreversible tier
38. Build ConfirmationUI: GNOME Shell extension (JavaScript) providing minimal confirmation overlay
39. Build AuditLogger: SHA-256 chained append-only log to ~/.local/share/osaima/audit.log
40. Build PromptSanitizer: strip instruction patterns from UNTRUSTED_CONTEXT before LLM injection

5.9 Phase 8 — MCP Server Integration (Week 15–16)

Model Context Protocol (MCP) servers extend OSAIMA's reach to external APIs through a unified interface, matching the OSAIMA presentation's 'Advanced API Ecosystem' objective.

MCP Server / Integration	What It Enables	Implementation
Spotify MCP	Voice control: play artist, skip, volume, playlist management	npm install @modelcontextprotocol/server-spotify
Gmail MCP	Voice compose, send, read, label, search emails	npm install @modelcontextprotocol/server-gmail
System Monitor MCP	CPU/RAM/disk/temp status via voice query	Custom Python MCP server using psutil
Network Manager MCP	WiFi connect/disconnect, VPN toggle via voice	Python MCP server over D-Bus NetworkManager
File System MCP	Voice-driven file operations with safety classification	Python MCP server with risk-tiered actions
Calendar MCP	Voice-scheduled events, reminders, free/busy check	npm install @modelcontextprotocol/server-gcal
Browser MCP	Open URLs, search, screenshot, form fill via voice	Playwright-based Python MCP server

- MCP host embedded in OSAIMA Python daemon; MCP servers spawned as child processes on demand
- LLM selects MCP tool calls via function-calling JSON schema in system prompt

- pip install mcp --break-system-packages; from mcp import ClientSession, StdioServerParameters

6. First-Ring Operations — OS Internals Access

This section addresses the advanced capability mentioned in the PDR brief: accessing file system internals and performing first-ring OS operations. These capabilities differentiate OSAIMA from app-level voice assistants.

6.1 File System Operations via Voice

Operation Category	Linux API / Tool	OSAIMA Integration
File search & indexing	locate (mlocate DB) / find / fd-find	Voice: 'find all Python files modified today in my projects'
File content search	ripgrep (rg) over PTY	Voice: 'search for TODO comments in this codebase'
File watching	inotifywait / pyinotify	Behavior store monitors ~/Documents for auto-indexing
Archive operations	libarchive Python bindings	Voice: 'extract this archive to the current project folder'
Disk usage analysis	du / ncdu over PTY	Voice: 'what is consuming the most disk space in my home directory'
Permissions management	os.chmod / subprocess ACL tools	Voice: 'make this script executable' — Reversible tier
Mount / unmount	udisksctl D-Bus interface	Voice: 'mount the USB drive' — surfaced via udisks2 D-Bus

6.2 Process & System Resource Control

- Process listing: Voice queries resolved via psutil Python library — cross-platform, zero root required
- Process signals: psutil.Process(pid).terminate() / .kill() — Reversible/Irreversible tier classification
- Service management: org.freedesktop.systemd1 D-Bus — start/stop/restart/status of any systemd unit
- Resource monitoring: psutil CPU, memory, disk, network I/O — fed to LLM as context for optimization queries
- cgroups v2 inspection: Read /sys/fs/cgroup/ for per-process resource accounting

6.3 /proc and /sys Filesystem Access

The Linux /proc and /sys virtual filesystems expose kernel state without requiring kernel modules. OSAIMA's action executor reads these for system introspection:

- /proc/cpuinfo — CPU architecture, cores, frequency (for LLM model selection optimization)
- /proc/meminfo — Available RAM (triggers model downgrade recommendation if < 6 GB free)
- /proc/$PID/maps — Process memory maps (for developer debugging workflows via voice)
- /sys/class/power_supply/ — Battery state on laptops (voice: 'what is my battery percentage')

- /sys/class/net/ — Network interface statistics (voice: 'show my network throughput')
- /sys/block/ — Block device info (voice: 'what storage devices are connected')

6.4 Kernel-Level Future: eBPF Wake-Word Detection

The research paper identifies eBPF-based wake-word detection as a future research direction. This represents true ring-0 adjacent operation — an eBPF program loaded into the kernel processes audio events with substantially lower user-space overhead than polling daemons. While outside the current project scope, the architecture is designed to support it:

- eBPF programs can be attached to perf_event sources; audio ringbuffer events from ALSA/PipeWire can trigger user-space callbacks
- Python eBPF libraries: bcc (BPF Compiler Collection) — pip install bcc --break-system-packages
- Prototype: bpf_program attached to snd_pcm_read() tracepoint triggers user-space daemon on audio activity
- This reduces continuous microphone polling CPU overhead from 5–15% to < 1%

7. OSAIMA vs. Existing Systems — Capability Matrix

System	Voice-First OS	Local LLM	LLM Workflows	Behavior Learn	OS-Level Ctrl	Priv
SmartLap	No	No	No	No	App	Full
Shree	No	No	No	No	Service	Full
Mycroft AI	No	No	No	No	Service	Partia
Rhasspy	No	Partial	No	No	Service	Full
OpenVoiceOS	No	No	No	No	Service	Full
local_llm_asst.	No	Yes	No	No	App	Full
Google Assistant	No	No	No	Pref	None	None
Amazon Alexa	No	No	No	Pref	None	None
OSAIMA (proposed)	YES	YES	YES	YES	OS	Full

8. MCP Server Architecture — Deep Dive

Model Context Protocol (MCP) is the architectural backbone for OSAIMA's API ecosystem. It provides a standardized interface through which the LLM can discover and invoke tools without tight coupling to any specific API.

8.1 MCP Communication Flow

[LLM generates tool-call JSON] → [MCP Host (OSAIMA daemon)] → [MCP Server (Spotify/Gmail/etc.)] → [External API / System] → [Result returned to LLM] → [LLM generates user response]

8.2 Custom System MCP Servers to Build

- osaima-filesystem-mcp: Wraps file operations with risk classification; exposes tools: read_file, write_file, list_directory, search_files, get_file_info
- osaima-process-mcp: Wraps psutil; exposes tools: list_processes, kill_process, get_system_stats, get_process_info
- osaima-dbus-mcp: Wraps common D-Bus interfaces; exposes tools: launch_app, control_media, manage_network, control_service
- osaima-terminal-mcp: Wraps PTY injection; exposes tools: run_command, run_in_active_terminal, get_command_output
- osaima-browser-mcp: Wraps Playwright; exposes tools: open_url, search_web, click_element, fill_form, take_screenshot

8.3 Building a Custom MCP Server (Python)

```
from mcp.server import Server\nfrom mcp.server.stdio import stdio_server\n\napp = Server('osaima-system')\n\n@app.list_tools()\nasync def list_tools(): return [Tool(name='get_cpu_usage', description='Get current CPU usage', inputSchema={...})]\n\n@app.call_tool()\nasync def call_tool(name, arguments):\n    if name == 'get_cpu_usage':\n        import psutil; return [TextContent(type='text', text=str(psutil.cpu_percent()))]
```

9. Testing & Evaluation Strategy

9.1 Benchmark Metrics

Metric	Target	Measurement Method
ASR Word Error Rate (dev vocab)	< 8% (Whisper), < 15% (Vosk)	Custom 500-utterance benchmark corpus with tech/Linux vocabulary
Intent Classification Accuracy	> 92%	Held-out 1,000-utterance test set from synthetic + manual data
System Command Latency (fast path)	< 200ms	time.perf_counter() wrapping dispatch; 100-run average
LLM Workflow Latency (8B model, CPU)	< 8 seconds (TTFT < 2s)	Streaming: Time-to-first-token + full generation time
Workflow Execution Success Rate	> 80% (no manual correction)	20 canonical developer tasks; 5 testers; success = correct result
False Wake-Word Rate	< 1 per hour idle	8-hour background noise recording; count false triggers
Prompt Injection Resistance	0 successful injections	Red-team dataset of 50 adversarial prompts

9.2 Integration Test Scenarios

- Voice → D-Bus: 'Open the Files app' → GNOME Files launches within 200ms
- Voice → LLM → PTY: 'Set up a Django project called myapp with my usual virtual env' → executes correctly in terminal

- Voice → LLM → AT-SPI: 'Fill in the login form with my credentials' → correct fields filled in browser
- Voice → RAG → LLM: 'How do I run the tests for this project' → reads project README, returns correct pytest command
- Voice → MCP → Spotify: 'Play something like what I was listening to yesterday' → Spotify plays similar track
- Behavior Learning: Same email task before/after 5 training interactions → measurably more consistent with user style
- Security: Red-team prompt injection via clipboard content → no unauthorized action executed

10. Development Roadmap — 18-Month Plan

Phase	Duration	Milestone	Deliverables
Phase 0: Foundation	Months 1–2	Dev environment + ASR working	Vosk daemon, openWakeWord, systemd services, basic PTY injection
Phase 1: Core Loop	Months 3–5	Voice → LLM → terminal working	Intent router, Ollama integration, basic D-Bus actions, risk classifier
Phase 2: OS Control	Months 6–8	Full D-Bus + AT-SPI integration	All D-Bus interfaces, AT-SPI automator, AppArmor profile, audit log
Phase 3: RAG + MCP	Months 9–11	Knowledge retrieval + API ecosystem	ChromaDB RAG, 5 custom MCP servers, MCP host, Gmail + Spotify
Phase 4: Learning	Months 12–14	Behavior store live	Style learner, workflow detector, personalized LLM prompts
Phase 5: Distro	Months 15–17	Custom Linux ISO	Ubuntu-based custom distro, voice-first boot, GNOME Shell extension
Phase 6: Evaluation	Month 18	PDR → final submission	Benchmark results, user study (n≥20), open-source release under GNU GPL

11. Hardware Requirements & Deployment Targets

Tier	RAM	CPU	GPU	Storage	Use Case
Minimum	8 GB	4-core x86_64 (2018+)	None	20 GB SSD	Phi-3-mini, Vosk-small, basic workflows
Recommended	16 GB	8-core x86_64 (2021+)	None	50 GB NVMe	Llama-3.1-8B, Vosk-large, full RAG
Optimal	32 GB	12-core x86_64 (2022+)	RTX 3060+	100 GB NVMe	Whisper-large, 8B GPU inference, all features
Embedded	4 GB	RPi 4 / Jetson Nano	Optional	32 GB SD	Phi-3-mini, Vosk-small, home automation focus

12. Expected Impact & Project Outcomes

Outcome	Target Metric	Basis
Productivity increase	75% reduction in task completion time for routine workflows	GitHub Copilot study: 55.8% for coding; extrapolated for OS tasks
Learning curve reduction	90% fewer commands to memorize for new Linux users	Natural language eliminates syntax memorization requirement
Error prevention	60% reduction in destructive command accidents	Risk-tiered confirmation prevents accidental rm -rf, overwrites
Privacy preservation	100% of audio and context data stays on-device	Structural — no cloud dependency in any component
Developer adoption potential	> 500K potential users (Linux dev population × interest %)	Stack Overflow survey: 40–55% devs on Linux; early adopter segment

13. Open Research Questions & Future Directions

- Multi-modal input: Combine voice with on-device screen vision (PaliGemma VLM) — 'summarize what is on screen right now'
- Federated behavior learning: Privacy-preserving style-vector aggregation across users via differential privacy
- eBPF wake-word detection: Kernel-resident audio event detection reducing continuous polling to < 1% CPU
- Standardized benchmarks: WER corpus for developer vocabulary; workflow success-rate benchmark for 20 canonical tasks
- Cross-DE portability: Unified voice-automation abstraction layer for GNOME, KDE Plasma, Sway, Hyprland
- Adversarial robustness: Formal threat model for LLM-driven OS agents; red-team evaluation protocol

14. Key References

The full reference list (37 sources) is available in the accompanying literature survey paper. Key references for this PDR:

41. Vosk Offline Speech Recognition: https://alphacephei.com/vosk/
42. Ollama Local LLM Runtime: https://github.com/ollama/ollama
43. llama.cpp C/C++ LLM Inference: https://github.com/ggml-org/llama.cpp
44. openWakeWord: https://github.com/dscripka/openWakeWord
45. ChromaDB Vector Store: https://www.trychroma.com/
46. Model Context Protocol Spec: https://modelcontextprotocol.io/

47. AT-SPI2 Accessibility API: https://www.freedesktop.org/wiki/Accessibility/AT-SPI2/
48. D-Bus Specification: https://dbus.freedesktop.org/doc/dbus-specification.html
49. Greshake et al., 'Indirect Prompt Injection' (2023): https://arxiv.org/abs/2302.12173
50. Market Research Future, Voice Assistant Market 2024–2030